

# Projeto A3 - Analise exploratoria e pre-processamento

**Problema:** prever se um cliente de telecomunicacoes cancelara o servico.

**Target:** Churn , onde Yes indica cancelamento e No indica permanencia.

Nesta etapa, vamos conhecer a base, corrigir problemas nos dados e preparar conjuntos de treino e teste. A modelagem sera feita na proxima etapa.

## Tecnica: importacao e configuracao das bibliotecas

Este bloco importa as ferramentas utilizadas no projeto:

- pandas e numpy : manipulacao e analise de dados;
- matplotlib e seaborn : criacao de graficos para a EDA;
- componentes do scikit-learn : separacao dos dados e pre-processamento.

O RANDOM\_STATE fixa a aleatoriedade para que a divisao dos dados possa ser reproduzida em outras execucoes.

```
In [1]: from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
```

# 1. Carregamento dos dados

O caminho abaixo funciona ao executar o notebook tanto na raiz do projeto quanto dentro da pasta `notebooks`.

**Tecnica utilizada:** leitura de dados tabulares com `pandas`.

O CSV e carregado em um DataFrame, estrutura que permite filtrar, transformar e analisar as linhas e colunas da base.

```
In [2]: possiveis_caminhos = [
        Path("../data/WA_Fn-UseC_-Telco-Customer-Churn.csv"),
        Path("data/WA_Fn-UseC_-Telco-Customer-Churn.csv"),
    ]

    caminho_dados = next((p for p in possiveis_caminhos if p.exists()), None)
    if caminho_dados is None:
        raise FileNotFoundError("Arquivo CSV nao encontrado na pasta data.")

    df = pd.read_csv(caminho_dados)
    print(f"Dimensoes da base: {df.shape[0]} linhas e {df.shape[1]} colunas")
    df.head()
```

Dimensoes da base: 7043 linhas e 21 colunas

Out[2]:

|  | customerID | gender | SeniorCitizen | Partner | Dependents | tenure | PhoneService | Multipl |
|--|------------|--------|---------------|---------|------------|--------|--------------|---------|
|--|------------|--------|---------------|---------|------------|--------|--------------|---------|

|   |            |        |   |     |    |    |     |    |
|---|------------|--------|---|-----|----|----|-----|----|
| 0 | 7590-VHVEG | Female | 0 | Yes | No | 1  | No  | No |
| 1 | 5575-GNVDE | Male   | 0 | No  | No | 34 | Yes |    |
| 2 | 3668-QPYBK | Male   | 0 | No  | No | 2  | Yes |    |
| 3 | 7795-CFOCW | Male   | 0 | No  | No | 45 | No  | No |
| 4 | 9237-HQITU | Female | 0 | No  | No | 2  | Yes |    |

5 rows × 21 columns



Cada linha representa um cliente. As features descrevem perfil, servicos contratados, forma de pagamento e valores cobrados.

**Tecnica utilizada:** inspecao estrutural dos dados.

`info()` mostra tipos e valores preenchidos. `describe()` gera estatísticas descritivas, ajudando a identificar escalas, categorias e possíveis problemas.

```
In [3]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   str
1   gender                7043 non-null   str
2   SeniorCitizen         7043 non-null   int64
3   Partner               7043 non-null   str
4   Dependents            7043 non-null   str
5   tenure                7043 non-null   int64
6   PhoneService          7043 non-null   str
7   MultipleLines         7043 non-null   str
8   InternetService       7043 non-null   str
9   OnlineSecurity        7043 non-null   str
10  OnlineBackup          7043 non-null   str
11  DeviceProtection      7043 non-null   str
12  TechSupport           7043 non-null   str
13  StreamingTV           7043 non-null   str
14  StreamingMovies       7043 non-null   str
15  Contract              7043 non-null   str
16  PaperlessBilling      7043 non-null   str
17  PaymentMethod         7043 non-null   str
18  MonthlyCharges        7043 non-null   float64
19  TotalCharges          7043 non-null   str
20  Churn                 7043 non-null   str
dtypes: float64(1), int64(2), str(18)
memory usage: 1.1 MB
```

```
In [4]: df.describe(include="all").T
```

Out[4]:

|                         | count  | unique | top              | freq | mean      | std       | min   | 25%  | 50%   |
|-------------------------|--------|--------|------------------|------|-----------|-----------|-------|------|-------|
| <b>customerID</b>       | 7043   | 7043   | 7590-VHVEG       | 1    | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>gender</b>           | 7043   | 2      | Male             | 3555 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>SeniorCitizen</b>    | 7043.0 | NaN    | NaN              | NaN  | 0.162147  | 0.368612  | 0.0   | 0.0  | 0.0   |
| <b>Partner</b>          | 7043   | 2      | No               | 3641 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>Dependents</b>       | 7043   | 2      | No               | 4933 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>tenure</b>           | 7043.0 | NaN    | NaN              | NaN  | 32.371149 | 24.559481 | 0.0   | 9.0  | 29.0  |
| <b>PhoneService</b>     | 7043   | 2      | Yes              | 6361 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>MultipleLines</b>    | 7043   | 3      | No               | 3390 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>InternetService</b>  | 7043   | 3      | Fiber optic      | 3096 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>OnlineSecurity</b>   | 7043   | 3      | No               | 3498 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>OnlineBackup</b>     | 7043   | 3      | No               | 3088 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>DeviceProtection</b> | 7043   | 3      | No               | 3095 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>TechSupport</b>      | 7043   | 3      | No               | 3473 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>StreamingTV</b>      | 7043   | 3      | No               | 2810 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>StreamingMovies</b>  | 7043   | 3      | No               | 2785 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>Contract</b>         | 7043   | 3      | Month-to-month   | 3875 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>PaperlessBilling</b> | 7043   | 2      | Yes              | 4171 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>PaymentMethod</b>    | 7043   | 4      | Electronic check | 2365 | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>MonthlyCharges</b>   | 7043.0 | NaN    | NaN              | NaN  | 64.761692 | 30.090047 | 18.25 | 35.5 | 70.35 |
| <b>TotalCharges</b>     | 7043   | 6531   | 20.2             | 11   | NaN       | NaN       | NaN   | NaN  | NaN   |
| <b>Churn</b>            | 7043   | 2      | No               | 5174 | NaN       | NaN       | NaN   | NaN  | NaN   |

## 2. Verificacao da qualidade dos dados

`TotalCharges` deveria ser numerica, mas foi carregada como texto porque possui espacos vazios. Esses espacos serao convertidos em valores ausentes.

**Tecnica utilizada:** limpeza e conversao de tipos.

Tambem verificamos duplicatas e valores ausentes. O parametro `errors="coerce"` transforma valores invalidos em `NaN`, permitindo que eles sejam tratados corretamente no pipeline.

```
In [5]: print("Linhas duplicadas:", df.duplicated().sum())
print("Valores ausentes antes da conversao:", df.isna().sum().sum())
print("Espacos vazios em TotalCharges:", df["TotalCharges"].str.strip().eq("").sum())

df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

print("Valores ausentes depois da conversao:")
df.isna().sum()[df.isna().sum() > 0]
```

```
Linhas duplicadas: 0
Valores ausentes antes da conversao: 0
Espacos vazios em TotalCharges: 11
Valores ausentes depois da conversao:
```

```
Out[5]: TotalCharges    11
dtype: int64
```

Os valores ausentes de `TotalCharges` serao preenchidos pela mediana dentro do pipeline. Isso evita usar informacoes do conjunto de teste durante o treinamento.

### 3. Analise do target

A base possui mais clientes que permaneceram do que clientes que cancelaram. Portanto, alem da acuracia, sera importante avaliar recall e F1-score.

**Tecnica utilizada:** analise de distribuicao da variavel alvo.

A contagem absoluta mostra quantos clientes existem em cada classe, enquanto a proporcao revela o desbalanceamento. Isso orienta a escolha das metricas de avaliacao usadas posteriormente.

```
In [6]: contagem_churn = df["Churn"].value_counts()
percentual_churn = df["Churn"].value_counts(normalize=True).mul(100).round(2)

pd.DataFrame({"Quantidade": contagem_churn, "Percentual": percentual_churn})
```

```
Out[6]:
```

|  | Quantidade | Percentual |
|--|------------|------------|
|--|------------|------------|

| Churn | Quantidade | Percentual |
|-------|------------|------------|
|-------|------------|------------|

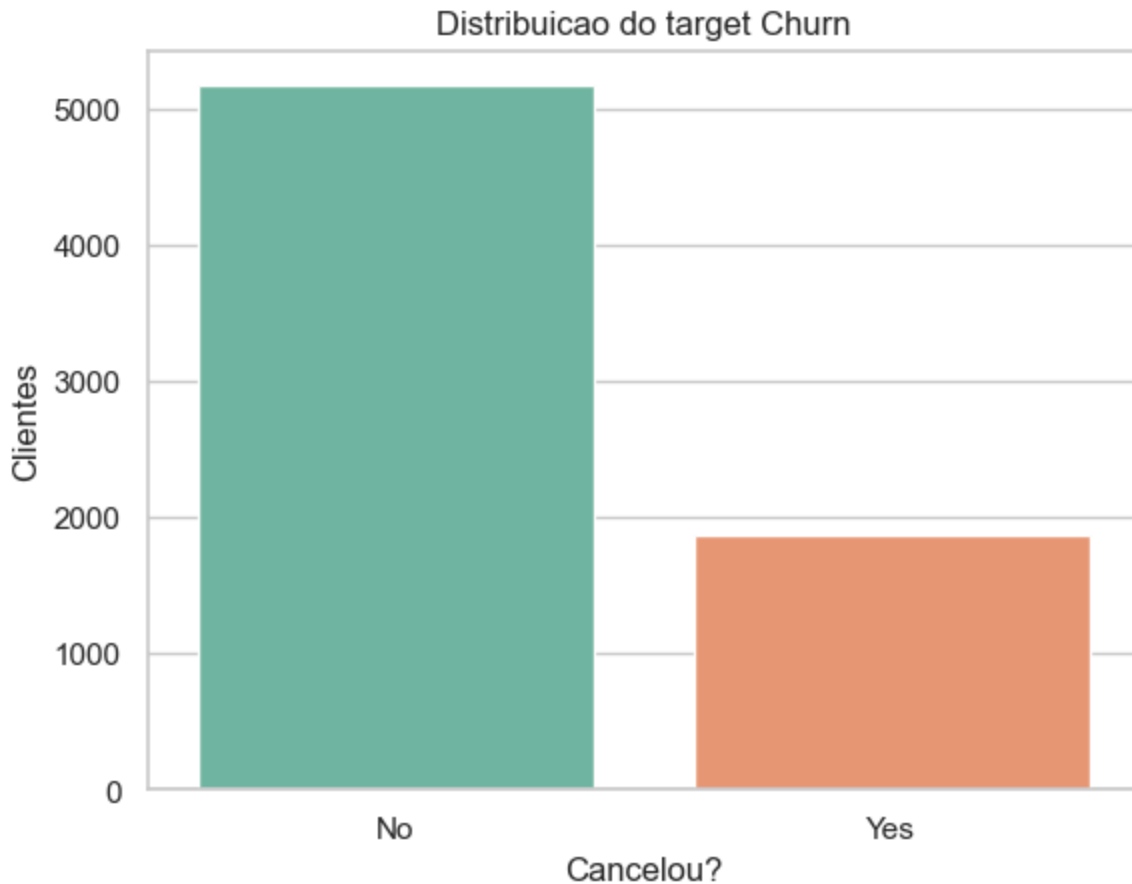
|    |      |       |
|----|------|-------|
| No | 5174 | 73.46 |
|----|------|-------|

|     |      |       |
|-----|------|-------|
| Yes | 1869 | 26.54 |
|-----|------|-------|

**Tecnica utilizada:** grafico de contagem.

O grafico facilita a comparacao visual entre as classes `Yes` e `No` . Ele evidencia que prever sempre a classe majoritaria poderia gerar uma acuracia aparentemente boa, mas sem identificar adequadamente os churns.

```
In [7]: ax = sns.countplot(data=df, x="Churn", hue="Churn", palette="Set2", legend=False)
ax.set(title="Distribuicao do target Churn", xlabel="Cancelou?", ylabel="Clientes")
plt.show()
```



## 4. Relacao das features com churn

Os graficos abaixo ajudam a identificar perfis com maior proporcao de cancelamento. A altura representa a taxa media de churn em cada categoria.

**Tecnica utilizada:** analise bivariada com graficos de barras.

O target e temporariamente convertido para 0 e 1. Assim, a media de `ChurnNumerico` em cada categoria representa diretamente a taxa de churn.

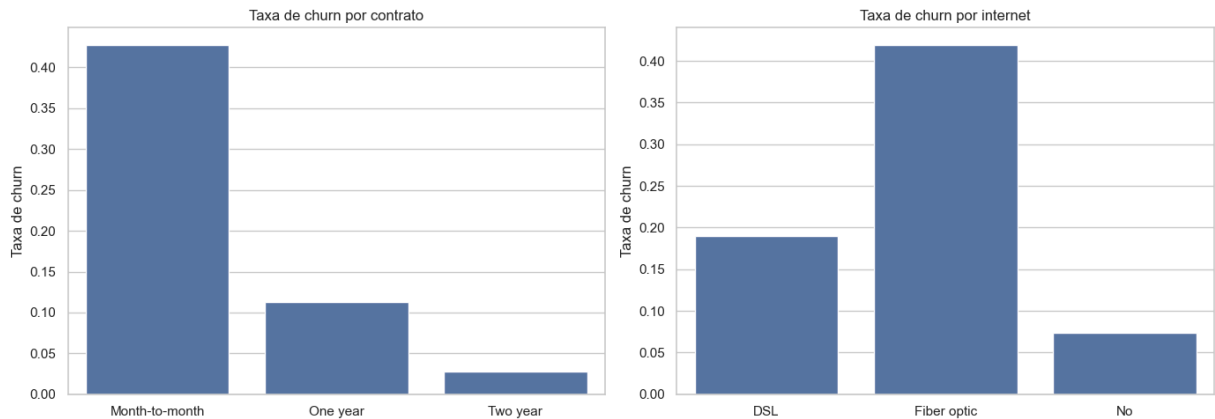
```
In [8]: df_eda = df.copy()
df_eda["ChurnNumerico"] = df_eda["Churn"].map({"No": 0, "Yes": 1})

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.barplot(data=df_eda, x="Contract", y="ChurnNumerico", ax=axes[0], errorbar=None)
sns.barplot(
```

```

data=df_eda,
x="InternetService",
y="ChurnNumerico",
ax=axes[1],
errorbar=None,
)
axes[0].set(title="Taxa de churn por contrato", ylabel="Taxa de churn", xlabel="")
axes[1].set(title="Taxa de churn por internet", ylabel="Taxa de churn", xlabel="")
plt.tight_layout()
plt.show()

```



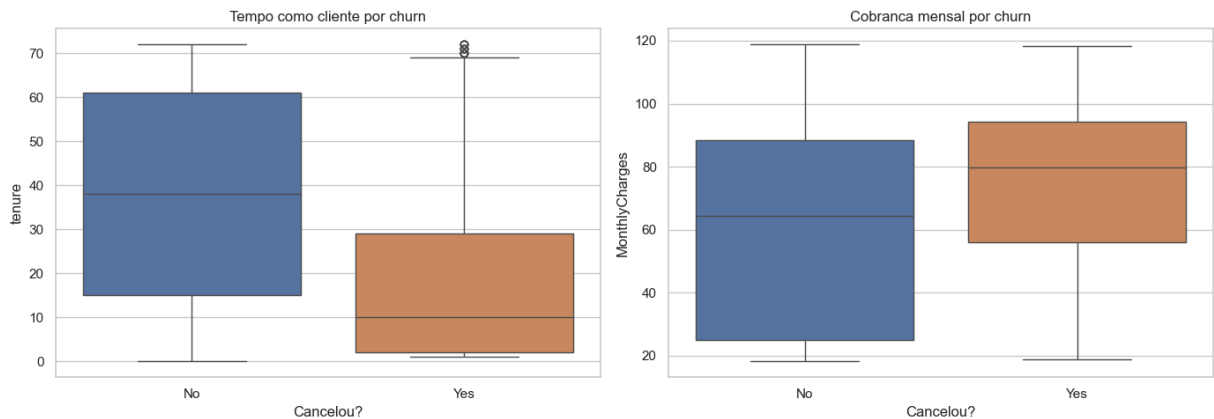
**Tecnica utilizada:** boxplot para comparar distribuicoes numericas.

O boxplot resume mediana, dispersao e valores extremos. Ele permite comparar o tempo como cliente ( `tenure` ) e a cobranca mensal entre clientes que cancelaram e clientes que permaneceram.

```

In [9]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.boxplot(data=df, x="Churn", y="tenure", ax=axes[0], hue="Churn", legend=False)
sns.boxplot(
    data=df,
    x="Churn",
    y="MonthlyCharges",
    ax=axes[1],
    hue="Churn",
    legend=False,
)
axes[0].set(title="Tempo como cliente por churn", xlabel="Cancelou?")
axes[1].set(title="Cobranca mensal por churn", xlabel="Cancelou?")
plt.tight_layout()
plt.show()

```



**Interpretacao inicial:** clientes com contrato mensal, menor tempo de permanencia e determinadas modalidades de internet aparentam apresentar maior risco de churn. Essa observacao devera ser confirmada pelos modelos.

## 5. Separacao entre features e target

`customerID` e apenas um identificador e nao ajuda a explicar o churn. O target e transformado em 0 e 1 para uso pelos classificadores.

**Tecnica utilizada:** divisao estratificada entre treino e teste.

Os dados de treino ensinam os modelos, enquanto os dados de teste simulam clientes ainda nao vistos. `stratify=y` preserva a proporcao de churn nos dois conjuntos, e `random_state` torna a divisao reproduzivel.

```
In [10]: X = df.drop(columns=["customerID", "Churn"])
y = df["Churn"].map({"No": 0, "Yes": 1})

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y,
)

print("Treino:", X_train.shape, "| Teste:", X_test.shape)
print("\nProporcao de churn no treino:")
print(y_train.value_counts(normalize=True).round(3))
print("\nProporcao de churn no teste:")
print(y_test.value_counts(normalize=True).round(3))
```



Treino: (5634, 19) | Teste: (1409, 19)

Proporcao de churn no treino:

Churn

0 0.735

1 0.265

Name: proportion, dtype: float64

Proporcao de churn no teste:

Churn

0 0.735

1 0.265

Name: proportion, dtype: float64

A separacao estratificada preserva aproximadamente a mesma proporcao de churn nos conjuntos de treino e teste.

## 6. Pipeline de pre-processamento

- Features numericas: valores ausentes preenchidos pela mediana e padronizacao com `StandardScaler`.
- Features categoricas: valor mais frequente para ausentes e codificacao one-hot.

O pipeline sera ajustado apenas durante o treinamento de cada modelo.

### Tecnicas utilizadas:

- **Imputacao pela mediana:** preenche valores numericos ausentes e sofre menos influencia de valores extremos;
- **Padronizacao:** transforma variaveis numericas para escalas comparaveis, importante principalmente para KNN e SVM;
- **One-hot encoding:** converte categorias em colunas binarias sem criar uma ordem artificial entre elas;
- **ColumnTransformer:** aplica tecnicas diferentes conforme o tipo da coluna;
- **Pipeline:** organiza as etapas e ajuda a evitar vazamento de dados.

```
In [11]: colunas_numericas = X.select_dtypes(include=np.number).columns.tolist()
colunas_categoricas = X.select_dtypes(exclude=np.number).columns.tolist()

pipeline_numerico = Pipeline(
    steps=[
        ("imputacao", SimpleImputer(strategy="median")),
        ("padronizacao", StandardScaler()),
    ]
)

pipeline_categorico = Pipeline(
    steps=[
        ("imputacao", SimpleImputer(strategy="most_frequent")),
        ("one_hot", OneHotEncoder(handle_unknown="ignore")),
    ]
)
```

```

    ]
)

preprocessador = ColumnTransformer(
    transformers=[
        ("numerico", pipeline_numerico, colunas_numericas),
        ("categorico", pipeline_categorico, colunas_categoricas),
    ]
)

print("Colunas numericas:", colunas_numericas)
print("Quantidade de colunas categoricas:", len(colunas_categoricas))

```

Colunas numericas: ['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges']  
 Quantidade de colunas categoricas: 15

## 7. Verificacao do pre-processamento

Esta transformacao e feita somente para verificar o resultado. Na etapa de modelagem, o `preprocessador` sera combinado com cada classificador em um unico pipeline.

`fit_transform` aprende os parametros usando apenas o treino e transforma os dados. No teste usamos somente `transform`, evitando que informacoes do teste influenciem o aprendizado.

```

In [12]: X_train_transformado = preprocessador.fit_transform(X_train)
        X_test_transformado = preprocessador.transform(X_test)

print("Formato original do treino:", X_train.shape)
print("Formato transformado do treino:", X_train_transformado.shape)
print("Formato transformado do teste:", X_test_transformado.shape)
print("Valores ausentes apos o pre-processamento:", np.isnan(X_train_transformado.d

```

Formato original do treino: (5634, 19)  
 Formato transformado do treino: (5634, 45)  
 Formato transformado do teste: (1409, 45)  
 Valores ausentes apos o pre-processamento: 0

## Conclusao da etapa

A base foi analisada e preparada para modelagem. Corrigimos `TotalCharges`, removemos o identificador, separamos treino e teste de forma estratificada e criamos um pipeline reutilizavel de pre-processamento.

**Proxima etapa:** treinar e avaliar KNN, Naive Bayes, SVM e Arvore de Decisao, seguidos pelo comite de classificadores.

# Projeto A3 - Modelagem e comite de classificadores

Nesta etapa, treinaremos e compararemos quatro classificadores:

- KNN (K-Nearest Neighbors)
- Naive Bayes
- SVM (Support Vector Machine)
- Arvore de Decisao

Depois, construiremos um comite por **votacao suave**. Nessa estrategia, a probabilidade prevista por cada modelo e considerada, e a classe com maior probabilidade media e escolhida.

## Tecnica: importacao e configuracao das bibliotecas

Este bloco importa os classificadores, as metricas, as ferramentas de pre-processamento e os recursos de visualizacao. O `RANDOM_STATE` torna reproduziveis a divisao dos dados e os algoritmos que possuem aleatoriedade.

```
In [1]: from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.base import clone
from sklearn.calibration import CalibratedClassifierCV
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import VotingClassifier
```

```
sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
```

# 1. Carregamento e preparacao dos dados

Repetimos a preparacao no notebook para que ele possa ser executado de forma independente. O mesmo `random_state` garante a mesma divisao da etapa anterior.

**Tecnicas utilizadas:** limpeza de dados, definicao de features/target e divisao estratificada. A divisao estratificada preserva a proporcao de churn, permitindo uma comparacao justa entre os modelos.

```
In [2]: possiveis_caminhos = [
        Path("../data/WA_Fn-UseC_-Telco-Customer-Churn.csv"),
        Path("data/WA_Fn-UseC_-Telco-Customer-Churn.csv"),
    ]
    caminho_dados = next((p for p in possiveis_caminhos if p.exists()), None)
    if caminho_dados is None:
        raise FileNotFoundError("Arquivo CSV nao encontrado na pasta data.")

    df = pd.read_csv(caminho_dados)
    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

    X = df.drop(columns=["customerID", "Churn"])
    y = df["Churn"].map({"No": 0, "Yes": 1})

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=RANDOM_STATE,
        stratify=y,
    )

    print("Treino:", X_train.shape, "| Teste:", X_test.shape)
```

Treino: (5634, 19) | Teste: (1409, 19)

**Tecnica utilizada:** pipeline de pre-processamento.

As variaveis numericas recebem imputacao pela mediana e padronizacao. As categoricas recebem imputacao pela moda e one-hot encoding. O resultado denso (`sparse_output=False`) e necessario para compatibilidade com o `GaussianNB`.

```
In [3]: colunas_numericas = X.select_dtypes(include=np.number).columns.tolist()
        colunas_categoricas = X.select_dtypes(exclude=np.number).columns.tolist()

        pipeline_numerico = Pipeline(
            steps=[
                ("imputacao", SimpleImputer(strategy="median")),
                ("padronizacao", StandardScaler()),
            ]
        )
```

```

)
pipeline_categorico = Pipeline(
    steps=[
        ("imputacao", SimpleImputer(strategy="most_frequent")),
        ("one_hot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ]
)
preprocessador = ColumnTransformer(
    transformers=[
        ("numerico", pipeline_numerico, colunas_numericas),
        ("categorico", pipeline_categorico, colunas_categoricas),
    ]
)

```

## 2. Definicao dos classificadores

Os hiperparametros abaixo formam uma configuracao inicial simples:

- KNN utiliza os 11 vizinhos mais proximos.
- Naive Bayes usa a distribuicao gaussiana.
- SVM usa kernel RBF e calibracao para gerar probabilidades para o comite.
- A Arvore tem profundidade limitada para reduzir overfitting.

Todos os modelos sao combinados ao mesmo preprocessador usando `Pipeline`.

### Como cada tecnica funciona:

- **KNN:** classifica pelo voto dos clientes mais proximos. Depende de padronizacao porque calcula distancias;
- **Naive Bayes:** estima probabilidades assumindo independencia entre as features. E simples, rapido e costuma apresentar recall elevado;
- **SVM:** procura uma fronteira que separe as classes. O kernel RBF permite fronteiras nao lineares, e a calibracao gera probabilidades para o comite;
- **Arvore de Decisao:** cria regras sucessivas e interpretaveis. Limitar a profundidade reduz o risco de memorizar os dados de treino;
- **clone:** cria copias independentes do preprocessador para cada modelo.

```

In [4]: classificadores = {
    "KNN": KNeighborsClassifier(n_neighbors=11),
    "Naive Bayes": GaussianNB(),
    "SVM": CalibratedClassifierCV(
        SVC(kernel="rbf", random_state=RANDOM_STATE),
        method="sigmoid",
        cv=3,
    ),
    "Arvore de Decisao": DecisionTreeClassifier(
        max_depth=6,
        min_samples_leaf=20,
        random_state=RANDOM_STATE,
    ),
}

```

```

}

modelos = {
    nome: Pipeline(
        steps=[
            ("preprocessamento", clone(preprocessador)),
            ("classificador", classificador),
        ]
    )
    for nome, classificador in classificadores.items()
}

list(modelos)

```

Out[4]: ['KNN', 'Naive Bayes', 'SVM', 'Arvore de Decisao']

### 3. Treinamento e avaliacao individual

Como o objetivo e identificar clientes que podem cancelar, consideramos **Churn = Yes** como classe positiva. Avaliaremos:

- **Acuracia:** proporcao total de previsoes corretas.
- **Precisao:** entre os clientes previstos como churn, quantos cancelaram.
- **Recall:** entre os clientes que cancelaram, quantos foram identificados.
- **F1-score:** equilibrio entre precisao e recall.

**Tecnica utilizada:** aprendizado supervisionado.

Cada modelo aprende usando pares de features e respostas conhecidas no conjunto de treino. Depois, realiza previsoes para clientes do conjunto de teste. A funcao

**calcular\_metricas** padroniza a avaliacao de todos os modelos.

```

In [5]: def calcular_metricas(y_verdadeiro, y_previsto):
        return {
            "Acuracia": accuracy_score(y_verdadeiro, y_previsto),
            "Precisao": precision_score(y_verdadeiro, y_previsto),
            "Recall": recall_score(y_verdadeiro, y_previsto),
            "F1-score": f1_score(y_verdadeiro, y_previsto),
        }

resultados = []
previsoes = {}

for nome, modelo in modelos.items():
    print(f"Treinando {nome}...")
    modelo.fit(X_train, y_train)
    y_pred = modelo.predict(X_test)
    previsoes[nome] = y_pred
    resultados.append({"Modelo": nome, **calcular_metricas(y_test, y_pred)})

resultados_df = (

```

```

pd.DataFrame(resultados)
.set_index("Modelo")
.sort_values("F1-score", ascending=False)
)
print(resultados_df.round(4))

```

Treinando KNN...

Treinando Naive Bayes...

Treinando SVM...

Treinando Arvore de Decisao...

|                   | Acuracia | Precisao | Recall | F1-score |
|-------------------|----------|----------|--------|----------|
| Modelo            |          |          |        |          |
| Naive Bayes       | 0.6948   | 0.4589   | 0.8369 | 0.5928   |
| KNN               | 0.7779   | 0.5822   | 0.5775 | 0.5799   |
| Arvore de Decisao | 0.7970   | 0.6679   | 0.4679 | 0.5503   |
| SVM               | 0.7935   | 0.6627   | 0.4519 | 0.5374   |

## 4. Matrizes de confusao dos modelos individuais

Cada matriz mostra:

- superior esquerdo: clientes sem churn classificados corretamente;
- superior direito: falsos alertas de churn;
- inferior esquerdo: churns nao identificados;
- inferior direito: churns identificados corretamente.

**Tecnica utilizada:** matriz de confusao.

Ela detalha os tipos de acerto e erro. Para retencao de clientes, o quadrante inferior esquerdo merece atencao, pois representa clientes que cancelaram, mas foram classificados como se fossem permanecer.

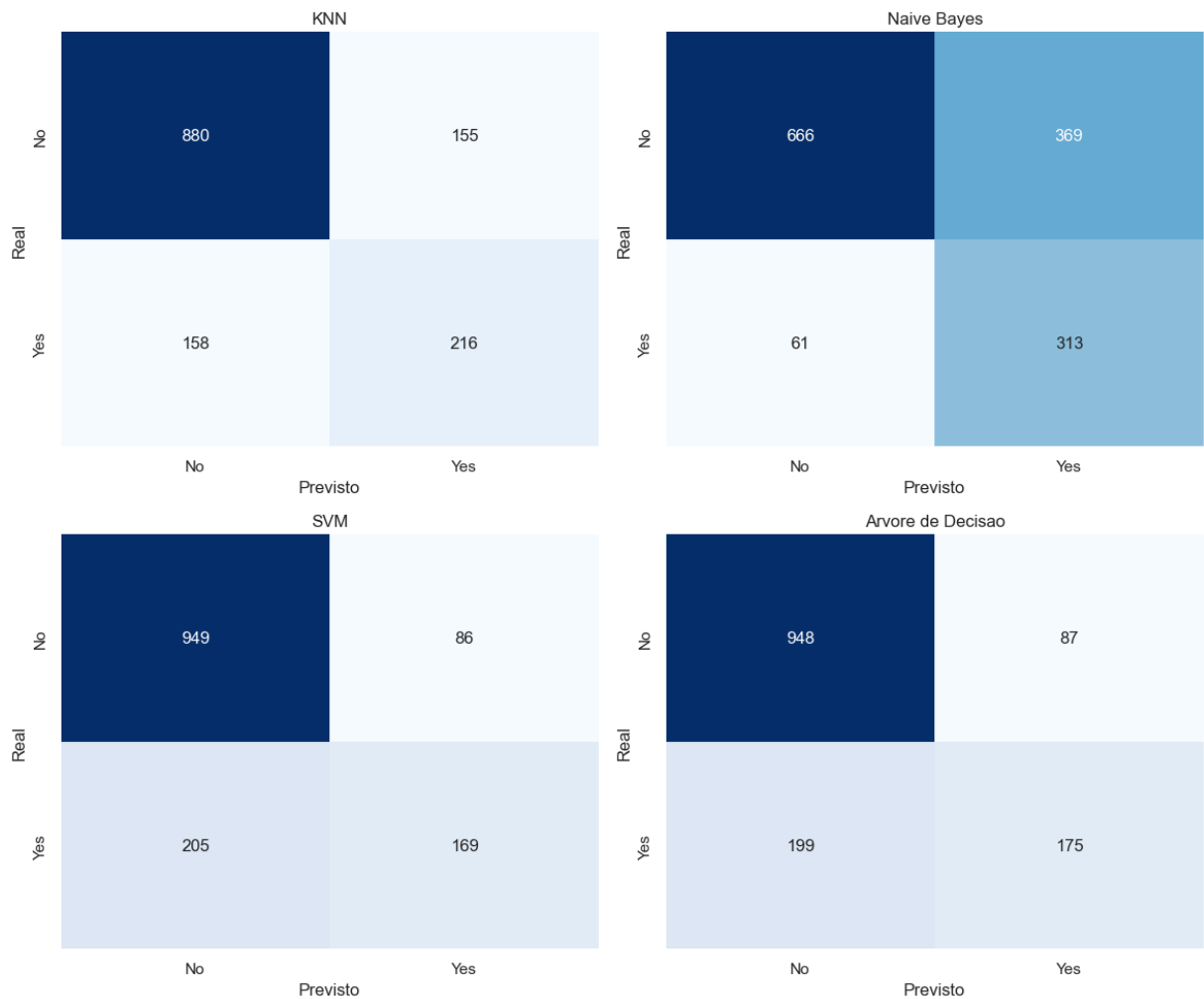
```

In [6]: fig, axes = plt.subplots(2, 2, figsize=(12, 10))

for ax, (nome, y_pred) in zip(axes.flat, previsoes.items()):
    matriz = confusion_matrix(y_test, y_pred)
    sns.heatmap(
        matriz,
        annot=True,
        fmt="d",
        cmap="Blues",
        cbar=False,
        ax=ax,
        xticklabels=["No", "Yes"],
        yticklabels=["No", "Yes"],
    )
    ax.set(title=nome, xlabel="Previsto", ylabel="Real")

plt.tight_layout()
plt.show()

```



## 5. Construcao do comite

O comite combina os quatro algoritmos por **votacao suave** ( `soft voting` ). Cada classificador informa a probabilidade de churn, e o comite calcula a media dessas probabilidades. Essa estrategia aproveita diferentes formas de aprendizado e considera o grau de confianca de cada modelo.

**Tecnica utilizada:** ensemble por votacao suave.

Um ensemble combina modelos com comportamentos diferentes para tentar obter uma decisao mais equilibrada. Diferentemente da votacao majoritaria, a votacao suave considera a confianca expressa pelas probabilidades.

```
In [7]: comite = VotingClassifier(
    estimators=[
        ("knn", clone(modelos["KNN"])),
        ("naive_bayes", clone(modelos["Naive Bayes"])),
        ("svm", clone(modelos["SVM"])),
        ("arvore", clone(modelos["Arvore de Decisao"])),
    ],
    voting="soft",
```



```

)

print("Treinando Comite (votacao suave)...")
comite.fit(X_train, y_train)
y_pred_comite = comite.predict(X_test)
previsoes["Comite"] = y_pred_comite

metricas_comite = calcular_metricas(y_test, y_pred_comite)
resultados_df.loc["Comite"] = metricas_comite
resultados_df = resultados_df.sort_values("F1-score", ascending=False)
print(resultados_df.round(4))

```

```
Treinando Comite (votacao suave)...
```

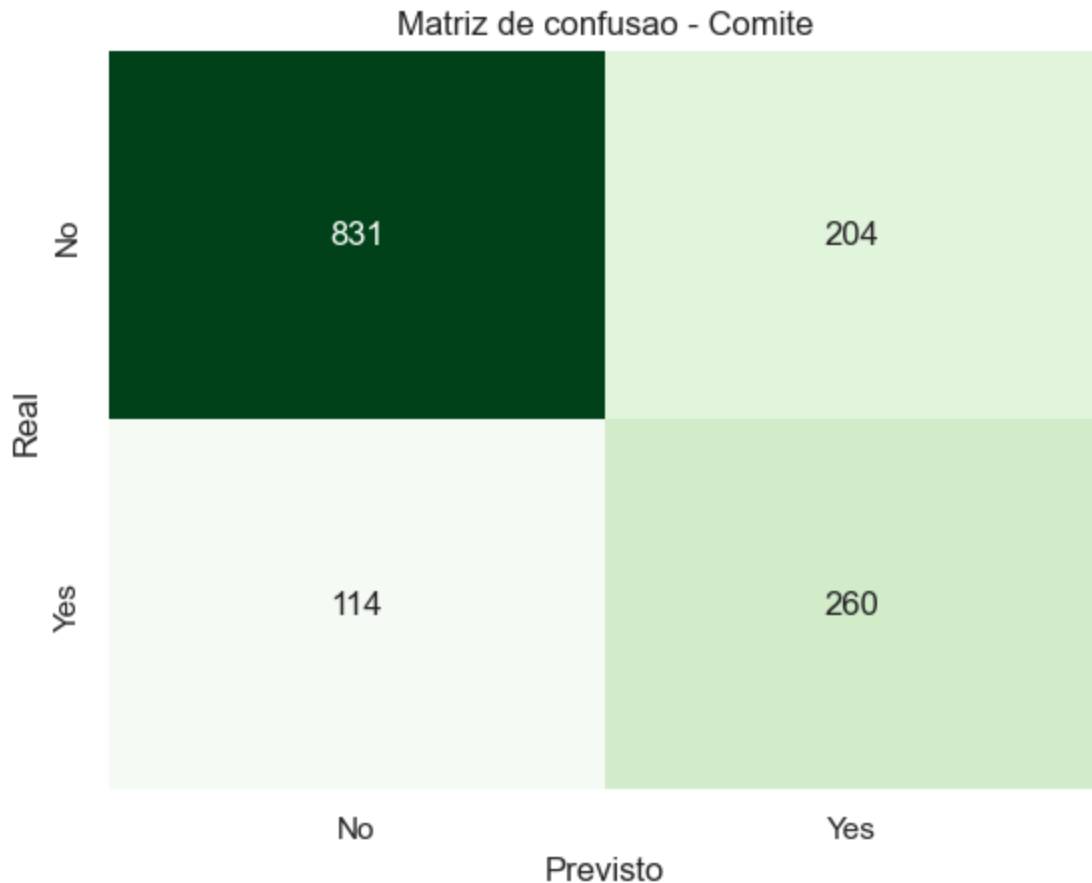
|                   | Acuracia | Precisao | Recall | F1-score |
|-------------------|----------|----------|--------|----------|
| Modelo            |          |          |        |          |
| Comite            | 0.7743   | 0.5603   | 0.6952 | 0.6205   |
| Naive Bayes       | 0.6948   | 0.4589   | 0.8369 | 0.5928   |
| KNN               | 0.7779   | 0.5822   | 0.5775 | 0.5799   |
| Arvore de Decisao | 0.7970   | 0.6679   | 0.4679 | 0.5503   |
| SVM               | 0.7935   | 0.6627   | 0.4519 | 0.5374   |

A matriz abaixo aplica a mesma tecnica de matriz de confusao ao comite. Ela permite verificar se a combinacao reduziu churns nao identificados sem gerar uma quantidade excessiva de falsos alertas.

```

In [8]: matriz_comite = confusion_matrix(y_test, y_pred_comite)
sns.heatmap(
    matriz_comite,
    annot=True,
    fmt="d",
    cmap="Greens",
    cbar=False,
    xticklabels=["No", "Yes"],
    yticklabels=["No", "Yes"],
)
plt.title("Matriz de confusao - Comite")
plt.xlabel("Previsto")
plt.ylabel("Real")
plt.show()

```



## 6. Comparacao final

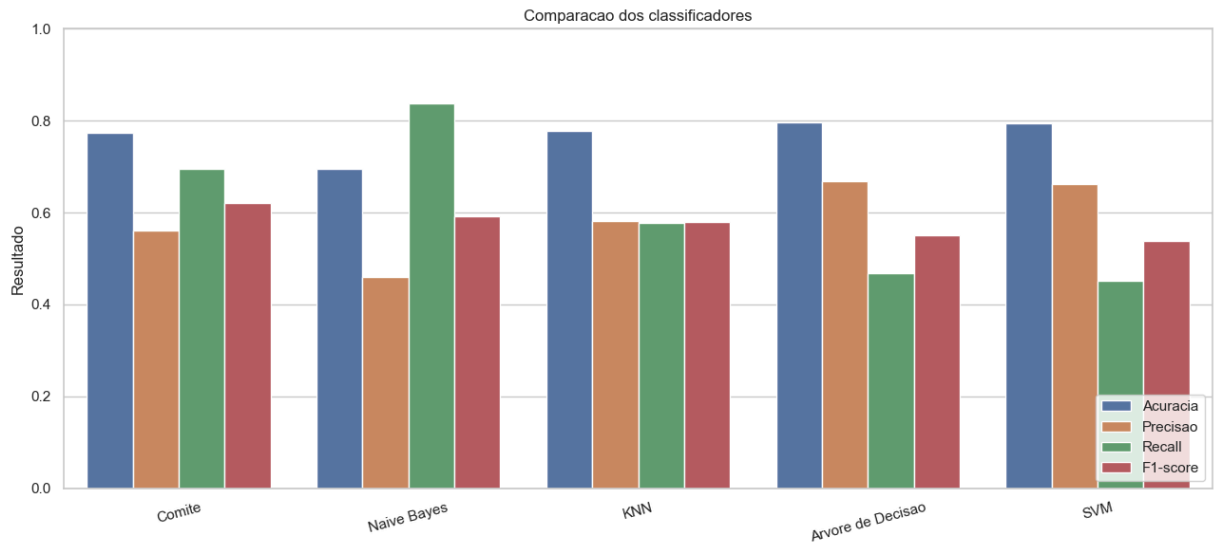
Como o target e desbalanceado, o F1-score e usado como criterio principal para comparar o equilibrio entre identificar churns e evitar falsos alertas.

**Tecnica utilizada:** comparacao visual de metricas.

O formato longo criado por `melt` permite colocar as quatro metricas lado a lado para cada modelo, facilitando a identificacao de seus pontos fortes e fracos.

```
In [9]: resultados_long = (
    resultados_df.reset_index()
    .melt(id_vars="Modelo", var_name="Metrica", value_name="Valor")
)

plt.figure(figsize=(13, 6))
ax = sns.barplot(data=resultados_long, x="Modelo", y="Valor", hue="Metrica")
ax.set(title="Comparacao dos classificadores", xlabel="", ylabel="Resultado")
ax.set_ylim(0, 1)
plt.xticks(rotation=15)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```



**Tecnica utilizada:** selecao do melhor modelo por metrica.

O melhor modelo individual e selecionado pelo maior F1-score e comparado ao comite. Essa regra responde objetivamente se o ensemble melhorou o equilibrio entre precisao e recall.

```
In [10]: melhor_individual = resultados_df.drop(index="Comite")["F1-score"].idxmax()
melhor_f1_individual = resultados_df.loc[melhor_individual, "F1-score"]
f1_comite = resultados_df.loc["Comite", "F1-score"]

print(f"Melhor modelo individual pelo F1-score: {melhor_individual}")
print(f"F1-score do melhor individual: {melhor_f1_individual:.4f}")
print(f"F1-score do comite: {f1_comite:.4f}")

if f1_comite > melhor_f1_individual:
    print("O comite apresentou melhora sobre o melhor modelo individual.")
else:
    print("O comite nao superou o melhor modelo individual pelo F1-score.")
```

Melhor modelo individual pelo F1-score: Naive Bayes

F1-score do melhor individual: 0.5928

F1-score do comite: 0.6205

O comite apresentou melhora sobre o melhor modelo individual.

## Conclusao da etapa

Os quatro classificadores individuais e o comite foram treinados e avaliados no mesmo conjunto de teste. O fato de um comite nao necessariamente superar o melhor modelo individual e um resultado valido: seu desempenho depende da diversidade e da qualidade dos classificadores que o compoem.

Na proxima etapa, os resultados serao interpretados em detalhes e usados na elaboracao do relatorio final.